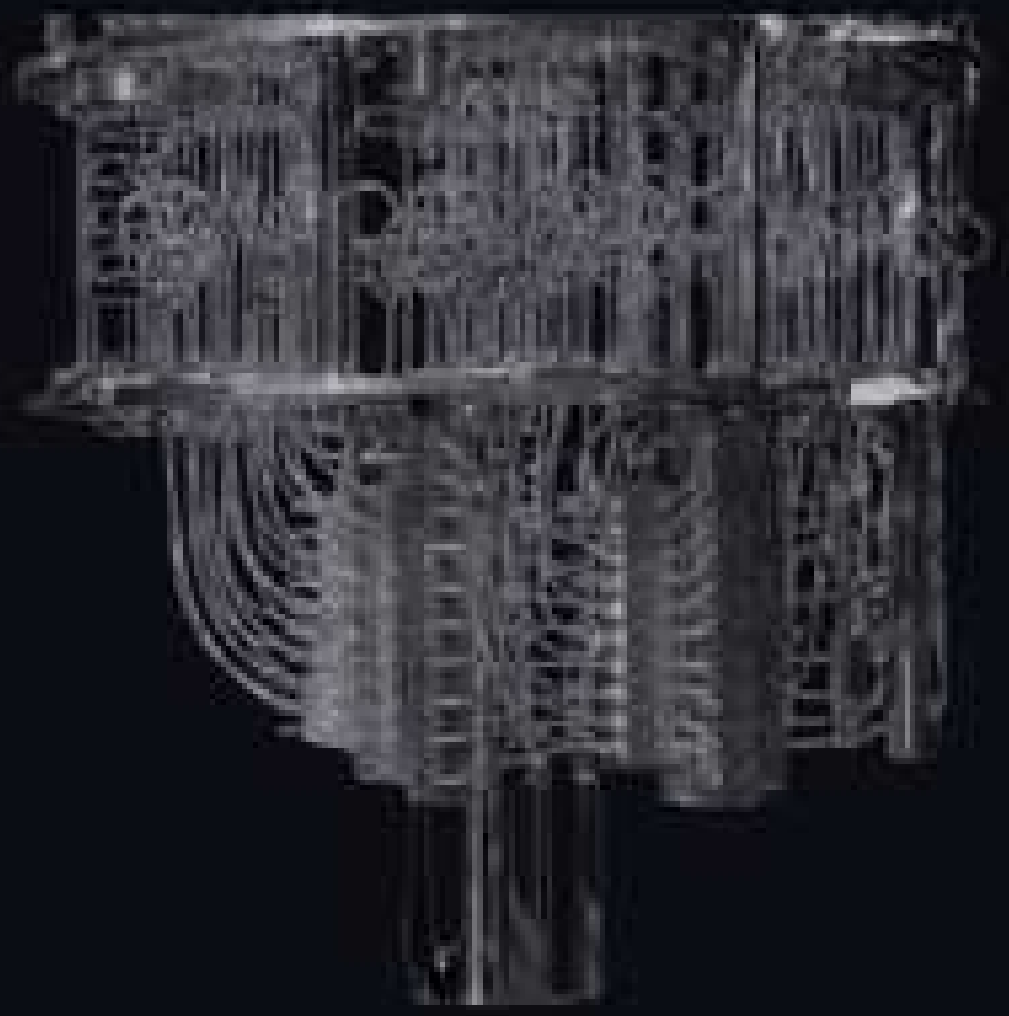
Starfighters — On the General Applicability of X-Wing

A comprehensive analysis of QSF and QSI combiners for post-quantum hybrid KEMs

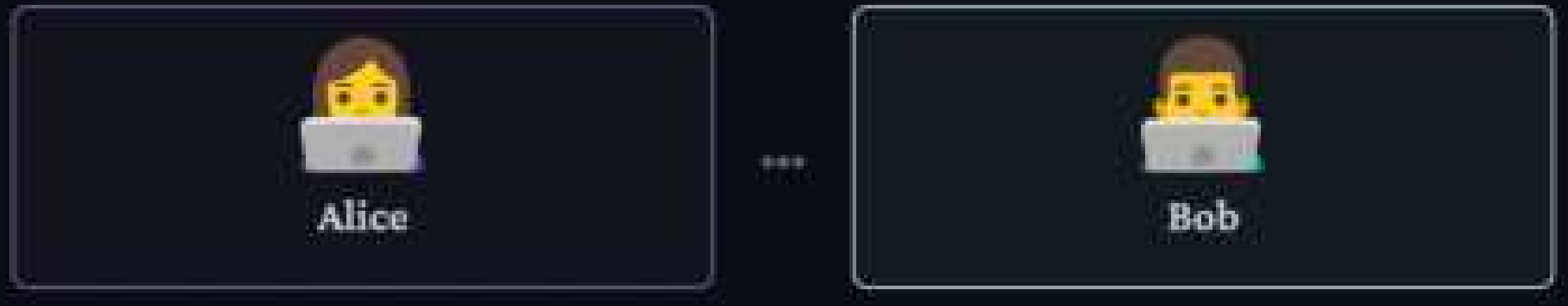
Deirdre Connolly · Kathrin Hövelmanns · Andreas Hülsing · Stavros Kousidis · Matthias Meijers

The Quantum Threat: Store Now, Decrypt Later

- Adversaries harvest encrypted traffic today for future quantum decryption
- Major investments in quantum computing — estimated over **$40 billion** globally
- Post-quantum schemes are newer, shorter track record than traditional elliptic curve/RSA
- PQ/T hybrids combine post-quantum + traditional: secure if *either* holds



Key Encapsulation Mechanisms (KEMs)



KeyGen() → (pk, sk)

$\mathsf{Encaps}(\mathsf{pk}) \to (k, c)$

Decaps(sk, c) $\rightarrow k$

Both share k

KEM Security: Indistinguishability

Given c, can you tell whether k is the *real* shared key or just *random*?

$$c, \ k$$

Real

actual shared key

vs

$$c, \ k_{\$}$$

Random

uniformly random key

Adversary can't tell which is which

But What If They Can Tamper?

Goal: capture malleability attacks

c'

Decapsulation Oracle

$m = d_{sk}(c')$

Adversary can **submit modified ciphertexts** and observe the result

This is IND-CCA security

Without it, ciphertext malleability enables practical key recovery (Bleichenbacher 1998)

Constructing Secure KEMs

Pretty much all PQ KEMs use FO: **ML-KEM, HQC, FrodoKEM, McEliece, NTRU**

The details vary.

FO in the Wild: An Actual Zoo

```
C0,C1 = C
C0 = list(C0)
C1 = list(C1)
m = decode.decode(C0,Gammaprime,params)
if m == False: return H(0,s,C1)
Ctprime = H(2,m)
if Ctprime != C1: return H(0,s,C1)
return H(1,m,C1)
```

Algorithm 18 ML-KEM.Decaps_internal(dk, c)

Uses the decapsulation key to produce a shared

Input: decapsulation key $dk \in \mathbb{B}^{768k+96}$

Input: ciphertext $c \in \mathbb{B}^{32(d_u k+d_v)}$

Output: shared secret key $K \in \mathbb{B}^{32}$

```
dk_PKE ← dk[0 : 384k]                    ▷ extrac
ek_PKE ← dk[384k : 768k + 32]
h ← dk[768k + 32 : 768k + 64]
z ← dk[768k + 64 : 768k + 96]
m' ← K-PKE.Decrypt(dk_PKE, c)
(K', r') ← G(m' || h)
K̄ ← J(z || c)
c' ← K-PKE.Encrypt(ek_PKE, m', r')
If c ≠ c' then
    K' ← K̄
end if
return K'
```

Some **hash c into the key**, **some don't**. Some use **key confirmation tags**. Some **use a salt**. Many **hash public key-related info**. Some do **pre-hashing**.

KEM Combiners

Secure if at least one combined component holds

KEM Combiners

KEM_0 + KEM_1 → Combined KEM

Secure if ***at least one*** component holds —
the other can be **completely broken**

Exactly what PQ/T hybrids need. Combiners differ in **what gets hashed**.

The IRTF CFRG Hybrid KEMs design team is standardizing these now.

$\mathsf{GHP}[\mathsf{KEM}_0, \mathsf{KEM}_1, \mathcal{H}]$ [Giacon, Heuer, Poettering 2018]

$\underline{\mathsf{Encaps}(\mathsf{pk})}$

$(k_0, c_0) \leftarrow \mathsf{KEM}_0.\mathsf{Encaps}(\mathsf{pk}_0)$

$(k_1, c_1) \leftarrow \mathsf{KEM}_1.\mathsf{Encaps}(\mathsf{pk}_1)$

$k \leftarrow \mathcal{H}(k_0,\ k_1,\ c_0,\ c_1)$

return $(k,\ (c_0, c_1))$

Hash both shared keys and both ciphertexts.

IND-CCA if either KEM is IND-CCA.

Large hash input — PQ ciphertexts are thousands of bytes.

$\mathsf{GHP}[\mathsf{KEM}_0, \mathsf{KEM}_1, H]$ [Giacon, Heuer, Poettering 2018]

$\underline{\mathsf{Encaps}(\mathsf{pk})}$

$(k_0, c_0) \leftarrow \mathsf{KEM}_0.\mathsf{Encaps}(\mathsf{pk}_0)$

$(k_1, c_1) \leftarrow \mathsf{KEM}_1.\mathsf{Encaps}(\mathsf{pk}_1)$

$k \leftarrow H(k_0, k_1, c_0, c_1)$

return $(k, (c_0, c_1))$

Hash **both shared keys** and **both ciphertexts**.

IND-CCA if either KEM is IND-CCA.

Large hash input — PQ ciphertexts are thousands of bytes.

QSF [Barbosa et al. 2024]

KeyGen()	Encaps(pk)	Decaps(c, sk)
$(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow\$ \mathsf{KEM.KeyGen}()$	$(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$	$(\mathsf{sk}_1, \mathsf{sk}_2, \mathsf{pk}_2) \leftarrow \mathsf{sk}$
$\mathsf{sk}_2 \xleftarrow{\$} \mathcal{E}_\lambda$	$k_1, c_1 \leftarrow\$ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$	$(c_1, c_2) \leftarrow c$
$\mathsf{pk}_2 \leftarrow \exp(g, \mathsf{sk}_2)$	$\mathsf{sk}_e \xleftarrow{\$} \mathcal{E}_\lambda$	$k_1 \leftarrow \mathsf{KEM.Decaps}(c_1, \mathsf{sk}_1)$
$\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$	$c_2 \leftarrow \exp(g, \mathsf{sk}_e)$	$k_2 \leftarrow \exp(c_2, \mathsf{sk}_2)$
$\mathsf{sk} \leftarrow (\mathsf{sk}_1, \mathsf{sk}_2, \mathsf{pk}_2)$	$k_2 \leftarrow \exp(\mathsf{pk}_2, \mathsf{sk}_e)$	if $k_1 = \perp$ then return $\perp$
return $(\mathsf{sk}, \mathsf{pk})$	$k \leftarrow H(k_1,\ k_2,\ c_2,\ \mathsf{pk}_2,\ \ell)$	$k \leftarrow H(k_1,\ k_2,\ c_2,\ \mathsf{pk}_2,\ \ell)$
	$c \leftarrow (c_1, c_2)$; return (k, c)	return k

Claim: c_1 KEM must be CCPA even if its IND-CCA fails

QSF [Barbosa et al. 2024]

KeyGen()	Encaps(pk)	Decaps(c, sk)
$(\mathsf{sk}_1, \mathsf{pk}_1) \leftarrow\$ \mathsf{KEM.KeyGen}()$	$(\mathsf{pk}_1, \mathsf{pk}_2) \leftarrow \mathsf{pk}$	$(\mathsf{sk}_1, \mathsf{sk}_2, \mathsf{pk}_2) \leftarrow \mathsf{sk}$
$\mathsf{sk}_2 \xleftarrow{\$} \mathcal{E}_\lambda$	$k_1, c_1 \leftarrow\$ \mathsf{KEM.Encaps}(\mathsf{pk}_1)$	$(c_1, c_2) \leftarrow c$
$\mathsf{pk}_2 \leftarrow \exp(g, \mathsf{sk}_2)$	$\mathsf{sk}_e \xleftarrow{\$} \mathcal{E}_\lambda$	$k_1 \leftarrow \mathsf{KEM.Decaps}(c_1, \mathsf{sk}_1)$
$\mathsf{pk} \leftarrow (\mathsf{pk}_1, \mathsf{pk}_2)$	$c_2 \leftarrow \exp(g, \mathsf{sk}_e)$	$k_2 \leftarrow \exp(c_2, \mathsf{sk}_2)$
$\mathsf{sk} \leftarrow (\mathsf{sk}_1, \mathsf{sk}_2, \mathsf{pk}_2)$	$k_2 \leftarrow \exp(\mathsf{pk}_2, \mathsf{sk}_e)$	if $k_1 = \perp$ then return $\perp$
return $(\mathsf{sk}, \mathsf{pk})$	$k \leftarrow H(k_1, k_2, c_2, \mathsf{pk}_2, \ell)$	$k \leftarrow H(k_1, k_2, c_2, \mathsf{pk}_2, \ell)$
	$c \leftarrow (c_1, c_2)$; return (k, c)	return k

Omitting c_1 *from H — KEM must be CCPA2 even if its IND-CCA fails*

C2PRI Ciphertext 2nd Preimage Resistance

Game $\mathsf{C2PRI}_{\mathsf{KEM}}(\mathcal{A})$

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}()$

$(k, c) \leftarrow \mathsf{Encaps}(\mathsf{pk})$

$c' \leftarrow \mathcal{A}(\mathsf{sk}, \mathsf{pk}, k, c)$

$\mathbf{return}\ \mathsf{Decaps}(\mathsf{sk}, c') = k \ \wedge\ c' \neq c$

Adversary gets *everything* — models a completely broken KEM.

C2PRI captures residual structural security from the hash functions.

Open Questions Before Starfighters

Can ML-KEM be replaced by **HQC, FrodoKEM, McEliece, or NTRU** in QSF?

Does **C2PRI** hold for all Fujisaki-Okamoto transform variants?

What about combining **two KEMs** instead of KEM + nominal group?

QSI

Quantum Superority Interceptor — a KEM-only combiner framework

$\mathsf{QSI}[\mathsf{KEM}_0, \mathsf{KEM}_1, H]$

KeyGen()	Encaps(pk)	Decaps(sk, c)
$(\mathsf{pk}_0, \mathsf{sk}_0) \leftarrow \mathsf{KEM}_0.\mathsf{KeyGen}()$	$(k_0, c_0) \leftarrow \mathsf{KEM}_0.\mathsf{Encaps}(\mathsf{pk}_0)$	$k_0 \leftarrow \mathsf{KEM}_0.\mathsf{Decaps}(\mathsf{sk}_0, c_0)$
$(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{KEM}_1.\mathsf{KeyGen}()$	$(k_1, c_1) \leftarrow \mathsf{KEM}_1.\mathsf{Encaps}(\mathsf{pk}_1)$	$k_1 \leftarrow \mathsf{KEM}_1.\mathsf{Decaps}(\mathsf{sk}_1, c_1)$
$\mathsf{pk} \leftarrow (\mathsf{pk}_0, \mathsf{pk}_1)$	$c \leftarrow (c_0, c_1)$	If $k_0 = \perp$ or $k_1 = \perp$
$\mathsf{sk} \leftarrow (\mathsf{sk}_0, \mathsf{sk}_1)$	$k \leftarrow H(k_0, k_1, \ell)$	return $\perp$
return $(\mathsf{pk}, \mathsf{sk})$	return (k, c)	$k \leftarrow H(k_0, k_1, \ell)$
		return k

At least one KEM must remain IND-CCA and the other must preserve CSPR to maintain IND-CCA of the combined scheme.

$\mathsf{QSI}[\mathsf{KEM}_0, \mathsf{KEM}_1, H]$

KeyGen()	Encaps(pk)	Decaps(sk, c)
$(\mathsf{pk}_0, \mathsf{sk}_0) \leftarrow \mathsf{KEM}_0.\mathsf{KeyGen}()$	$(k_0, c_0) \leftarrow \mathsf{KEM}_0.\mathsf{Encaps}(\mathsf{pk}_0)$	$k_0 \leftarrow \mathsf{KEM}_0.\mathsf{Decaps}(\mathsf{sk}_0, c_0)$
$(\mathsf{pk}_1, \mathsf{sk}_1) \leftarrow \mathsf{KEM}_1.\mathsf{KeyGen}()$	$(k_1, c_1) \leftarrow \mathsf{KEM}_1.\mathsf{Encaps}(\mathsf{pk}_1)$	$k_1 \leftarrow \mathsf{KEM}_1.\mathsf{Decaps}(\mathsf{sk}_1, c_1)$
$\mathsf{pk} \leftarrow (\mathsf{pk}_0, \mathsf{pk}_1)$	$c \leftarrow (c_0, c_1)$	If $k_0 = \perp$ or $k_1 = \perp$
$\mathsf{sk} \leftarrow (\mathsf{sk}_0, \mathsf{sk}_1)$	$k \leftarrow H(k_0, k_1, \ell)$	return $\perp$
return $(\mathsf{pk}, \mathsf{sk})$	return (k, c)	$k \leftarrow H(k_0, k_1, \ell)$
		return k

***Only hashes keys** and a label — no ciphertexts in the hash input*

QSI IND-CCA Security

Either KEM provides IND-CCA; the other only needs C2PRI

PQ KEM breaks

→ secure from traditional KEM

Traditional KEM breaks

→ secure from PQ KEM

Does not rely on (Q)ROM — standard model PRF suffices

QSF vs QSI

PROPERTY	QSF	QSI
Constituents	KEM + nominal group	KEM + KEM
Hash input	k_1, k_2, c_1, pk_2, t	k_1, k_2, t
IND-CCA	✓ (IND-CCA + C2PRI/SDH)	✓ (IND-CCA + C2PRI)
Standard model	Needs ROM for SDH case	✓ sKPRF suffices
Binding properties	✗ Fails CT-K	✓ All preserved
Multi-user security	Via pk in hash + inherited from KEM	Inherits from constituents

Binding properties (HON-BIND, LEAK-BIND, MAL-BIND) unaltered for both — see paper.

Which KEMs are C2PRI?

Ali Pujjadi-Okamoto variants — every practical PQ KEM

Explicit rejection

Invalid ciphertexts fail → second preimage for H

Implicit rejection

Pseudorandom on failure → still needs preimage for H or F

Salted variants

Random salt enlarges challenge space → identical bounds

Key confirmation

Appended tag → same reduction applies

C2PRI always reduces to second-preimage resistance of the KEM's hash/KDF

C2PRI for Real-World KEMs

ML-KEM

Implicit rejection

✅

HQC

Hybrid variant

✅

FrodoKEM

Hybrid variant

✅

Classic McEliece

Key confirmation

✅

sntrup

Self-verifying ciphertext

✅

NTRU

Implicit rejection

✅

Every standardized PQ KEM is C2PRI — QSF and QSI work with all of them

QSI for PQ/PQ Hybrids

$$\text{PQ KEM}_0 \quad + \quad \text{PQ KEM}_1$$

QSF needs ECDH — QSI combines any two KEMs

Hedge against a single PQ assumption failing

No classical dependency — **purely post-quantum**

Either KEM can provide IND-CCA

What We Show

1. **C2PRI holds for every FO variant** — QSF works with every standardized PQ KEM, not just ML-KEM

2. **QSI: a new KEM-only combiner** — combines any two IND-CCA and C2PRI KEMs with just $k = H(k_0, k_1, \ell)$

3. **C2PRI is why it all works** — hash-based structural security that survives even total IND-CCA failure

Many standardized PQ KEMs work with QSF or QSI today.

Starfighters — On the General Applicability of X-Wing

A comprehensive analysis of QSF and QSI combiners for post-quantum hybrid KEMs

Deirdre Connolly · **Kathrin Hövelmanns** · Andreas Hülsing · Stavros Kousidis · Matthias Meijers

ePrint 2026/1397

Bonus Slides

KEM IND-CCA

Game IND-CCA$_{\mathsf{KEM}}(\mathcal{A}, b)$

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}()$

$(k^*, c^*) \leftarrow \mathsf{Encaps}(\mathsf{pk})$

if b: $k^* \xleftarrow{\$} \mathcal{K}$

$b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Decaps}}}(\mathsf{pk}, c^*, k^*)$

return b'

$\mathcal{O}_{\mathsf{Decaps}}(c)$

if $c = c^*$: return $\perp$

$k \leftarrow \mathsf{Decaps}(\mathsf{sk}, c)$

return k

Adversary distinguishes real shared key from random, with a decapsulation oracle (excluding the challenge).

Do We Need to Hash the Public Key?

For PO-based KEMs, multi-user security is achieved by hashing **pk** into the shared key. One might think QSI needs the same — changing the derivation to:

$$H(k_0,\ k_1,\ c_0,\ c_1,\ \mathbf{pk}_0,\ \mathbf{pk}_1,\ \ell)$$

- This would **significantly increase hash input size** — especially with large PQ ciphertexts and public keys.
- Fortunately, **QSI inherently preserves multi-user/challenge IND-CCA** here in continuous — no additional binding needed.
- The proof carries through because multi-user CPA1, multi-user IND-CCA, and multi-user IAPM each compose naturally in the game sequence.

Hashing **pk** *can improve some binding properties, but at the cost of larger hash inputs — the trade-off is context-dependent.*

$\mathsf{T}[\mathsf{PKE}, G]$ — Derandomization

$\underline{\mathsf{Enc}'(\mathsf{pk}, m)}$

$c \leftarrow \mathsf{Enc}(\mathsf{pk}, m;\ G(m))$

return c

$\underline{\mathsf{Dec}'(\mathsf{sk}' := (\mathsf{pk}, \mathsf{sk}),\ c)}$

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$

if $m = \perp$ or $c \neq \mathsf{Enc}'(\mathsf{pk}, m)$

  return $\perp$

return m

- Decrypt-then-re-encrypt: randomness is fixed from the message via hash G — makes encryption deterministic.
- Ciphertext validity is checked via **rigidity**: $\mathsf{Enc}(\mathsf{pk}, \mathsf{Dec}(\mathsf{sk}, c)) = c$.

$\mathsf{U}^{\perp}$ — Explicit Rejection

$\underline{\mathsf{Encaps}(\mathsf{pk})}$

$m \xleftarrow{\$} \mathcal{M}$

$c \leftarrow \mathsf{Enc}(\mathsf{pk}, m)$

$k \leftarrow H(m, c)$

return (k, c)

$\underline{\mathsf{Decaps}^{\perp}(\mathsf{sk}, c)}$

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$

if $m = \perp$

  return $\perp$

return $k \leftarrow H(m, c)$

$\underline{\mathsf{Decaps}^{\perp}_{m}(\mathsf{sk}, c)}$

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$

if $m = \perp$

  return $\perp$

return $k \leftarrow H(m)$

$\mathsf{U}^{\perp}$ *hashes* (m, c); *variant* $\mathsf{U}^{\perp}_{m}$ *hashes* m *only. Both return* $\perp$ *on failure.*

$\mathsf{U}^{\not\perp}$ — Implicit Rejection

$\underline{\mathsf{Decaps}^{F}(\mathsf{sk}' := (\mathsf{sk}, \mathsf{fk}),\ c)}$

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$

$\text{if } m = \perp$

$\quad \text{return } k \leftarrow F(\mathsf{fk}, c)$

$\text{return } k \leftarrow H(m, c)$

$\underline{\mathsf{Decaps}^{F}_{\mathsf{m}}(\mathsf{sk}' := (\mathsf{sk}, \mathsf{fk}),\ c)}$

$m \leftarrow \mathsf{Dec}(\mathsf{sk}, c)$

$\text{if } m = \perp$

$\quad \text{return } k \leftarrow F(\mathsf{fk}, c)$

$\text{return } k \leftarrow H(m)$

- **Never returns** $\perp$ — invalid ciphertexts get a pseudorandom key $F(\mathsf{fk}, c)$
- Used by MLKEM, HPKE, TLS 1.3 PQ, Classic McEliece, BTRKEM — the dominant variant in practice
- CCA2 holds even when $F \approx H$ (Theorem 10)